

✓ Introduction to CUDA programming on Google Colab

This lesson shows how to run CUDA programs (.cu) on Colab.

✓ Enabling GPU support on Runtime

On the menu tab, "Runtime" => "Change runtime type". Select option "with GPU".

Check GPU card with *nvidia-smi* command

```
!nvidia-smi
```

```
⚡ Fri Jul 5 09:48:40 2024
```

NVIDIA-SMI 535.104.05				Driver Version: 535.104.05		CUDA Version: 12.2		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	MIG M.
0	Tesla T4		Off	00000000:00:04:0	Off		0	
N/A	50C	P8	10W / 70W		0MiB / 15360MiB	0%	Default	N/A

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	
	ID	ID				Usage	
No running processes found							

In this case the GPU is a Tesla T4 with 16GB of memory.

✓ GPU programming in CUDA - Hello World example

```
%%sh
cat > hello.cu << EOF
#include <stdio.h>

#define NUM_BLOCKS 16
#define BLOCK_WIDTH 1

__global__ void hello()
{
    printf("Hello world! I'm a thread in block %d\n", blockIdx.x);
}

int main(int argc, char **argv)
{
    // launch the kernel
    hello<<NUM_BLOCKS, BLOCK_WIDTH>>>();

    // force the printf()s to flush
    cudaDeviceSynchronize();

    printf("That's all!\n");

    return 0;
}
EOF
nvcc -o hello_cuda hello.cu && ./hello_cuda
```

```
⚡ Hello world! I'm a thread in block 1
Hello world! I'm a thread in block 11
Hello world! I'm a thread in block 4
Hello world! I'm a thread in block 14
Hello world! I'm a thread in block 6
Hello world! I'm a thread in block 9
Hello world! I'm a thread in block 3
Hello world! I'm a thread in block 0
Hello world! I'm a thread in block 13
Hello world! I'm a thread in block 10
Hello world! I'm a thread in block 5
Hello world! I'm a thread in block 8
```

```

Hello world! I'm a thread in block 15
Hello world! I'm a thread in block 12
Hello world! I'm a thread in block 2
Hello world! I'm a thread in block 7
That's all!

```

✓ GPU programming in CUDA - Array Sum example

```

%%sh
cat > arraysum.cu << EOF
/*
 * Adapted from:
 *
 * NVIDIA programming guide example
 * CUDA C++ Programming Guide PG-02829-001_v11.6 | page 22
 */

#include <stdio.h>
#include <cuda.h>

// Device code
__global__ void VecAdd(float* A, float* B, float* C, int N)
{
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N)
        C[i] = A[i] + B[i];
}

// Host code
int main()
{
    int N = 10000;
    size_t size = N * sizeof(float);

    // Allocate input vectors h_A and h_B in host memory
    float* h_A = (float*)malloc(size);
    float* h_B = (float*)malloc(size);
    float* h_C = (float*)malloc(size);

    // Initialize input vectors
    for(int i=0; i<N; i++)
    {
        h_A[i]=float(i+1);
        h_B[i]=20000.0;
    }

    // Allocate vectors in device memory
    float* d_A;
    cudaMalloc(&d_A, size);
    float* d_B;
    cudaMalloc(&d_B, size);
    float* d_C;
    cudaMalloc(&d_C, size);

    // Copy vectors from host memory to device memory
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // Invoke kernel
    int threadsPerBlock = 256;
    int blocksPerGrid =
        (N + threadsPerBlock - 1) / threadsPerBlock;

    VecAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

    // Copy result from device memory to host memory
    // h_C contains the result in host memory
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    // Free device memory
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    // Free host memory

    // Show Sum result on array c
    for(int i=0; i<N; i++)
    {
        printf("Result C[%d] = %f\n", i, h_C[i]);
    }
}

```

```
    if (i == 5) i = 0;  
    }  
}  
EOF  
nvcc -o arraysum arraysum.cu && ./arraysum
```

↔ Result C[0] = 20001.000000
Result C[1] = 20002.000000
Result C[2] = 20003.000000
Result C[3] = 20004.000000
Result C[4] = 20005.000000
Result C[5] = 20006.000000
Result C[9996] = 29997.000000
Result C[9997] = 29998.000000
Result C[9998] = 29999.000000
Result C[9999] = 30000.000000